

# Concepts

This page explains the architectural ideas behind DataSyncX: why the framework exists, how ETL work is divided, and which orchestration components coordinate task execution. For hands-on usage and component-specific parameters, use the dedicated guides for readers, writers, processors, and the API reference.

## Architecture

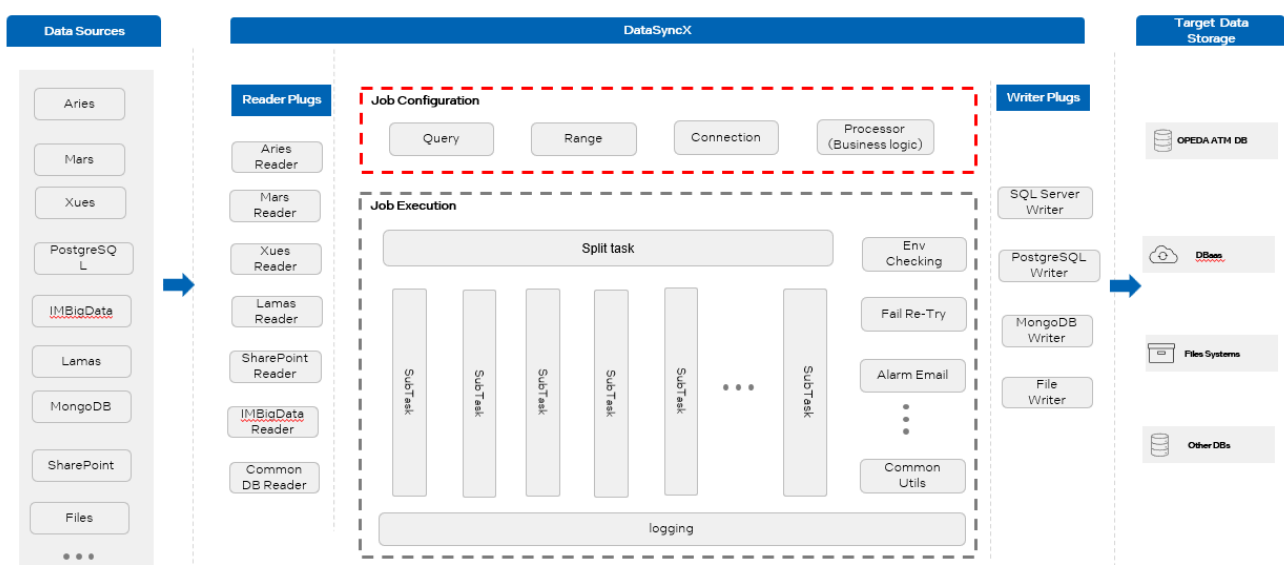
### Pipeline model

DataSyncX separates an ETL workflow into reusable stages:

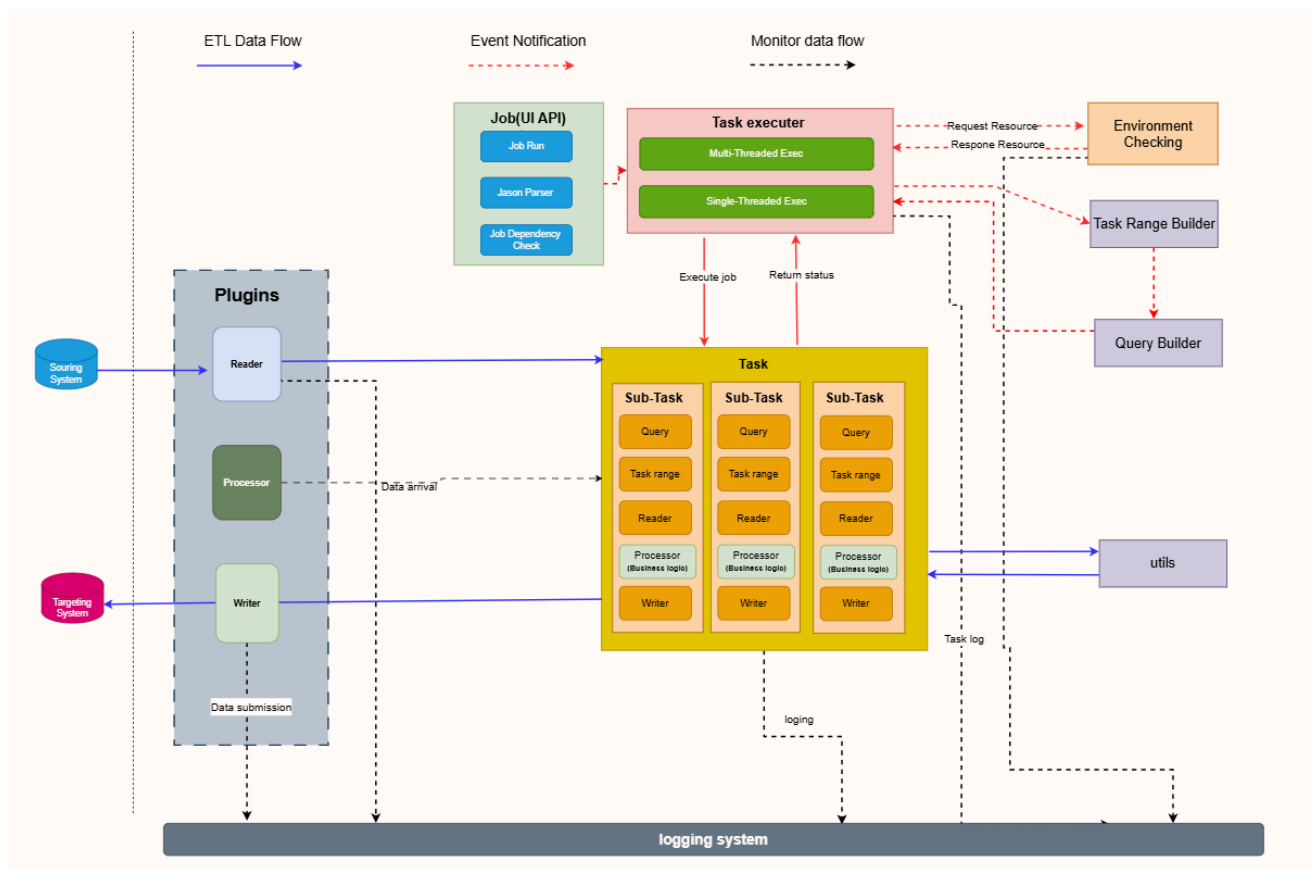
- Reader: extracts data from a source system.
- Processor: applies business logic or data transformations.
- Writer: loads the result into a target system.
- Orchestration components: build task ranges, generate queries, and execute tasks.

This separation keeps business logic isolated from connection handling and task scheduling.

### Architecture



### Design details



## ETL task processing flow

- An ETL task is composed of multiple independent components.
- Job Scheduler: After the task is triggered, it checks jobs, schedulers, dependencies, etc.
- Task List Generator: Creates queries based on task range.

# Components

## Core

The core orchestration layer coordinates how pipeline components run together:

- **Task:** the runnable ETL unit that combines a reader, a writer, and an optional processor.
- **TaskExecutor:** runs tasks in serial or parallel execution modes.
- **TaskRangeBuilder:** builds execution ranges such as time windows or token-based batches.
- **QueryBuilder:** injects runtime parameters into query templates.

## Utils

Supporting utilities provide configuration management, logging, notifications, and related shared helpers used by tasks and components.

# Component guides

- Reader usage and supported sources: [readers.md](#)

- Writer usage and supported targets: [writers.md](#)
- Processor usage and extension pattern: [processors.md](#)
- API reference for core modules and component classes: [api/index.md](#)